

SOFTWARE TESTING – BÀI TẬP 04 (VLU)

UNIT TESTING WITH JUNIT

1. Giới thiệu JUnit

JUnit là framework dùng để viết và thực thi Unit Test trong Java. JUnit giúp:

- Tự động hóa kiểm thử
- Phát hiện lỗi sớm
- Đảm bảo chất lượng mã nguồn

2. Cấu trúc một Unit Test trong JUnit

2.1 Annotation trong JUnit

Annotation Ý nghĩa

@Test Đánh dấu phương thức là test case

@Before Chạy trước mỗi test

@After Chạy sau mỗi test

@BeforeClass Chạy một lần trước tất cả test

@AfterClass Chạy một lần sau tất cả test

3. Các phương thức Assert

Phương thức Mô tả

assertEquals So sánh giá trị mong đợi và thực tế

assertTrue Kiểm tra điều kiện đúng

assertFalse Kiểm tra điều kiện sai

assertNull Kiểm tra giá trị null

assertNotNull Kiểm tra không null

4. Ví dụ Unit Test với JUnit

4.1 Hàm cần kiểm thử

java

```
public int add(int a, int b) {  
    return a + b;  
}
```

4.2 Test case

java

@Test

```
public void testAdd() {  
    assertEquals(5, add(2, 3));  
}
```

5. Parameterized Test

Khái niệm

Parameterized Test cho phép chạy nhiều bộ dữ liệu cho cùng một test case.

Ví dụ

java

@ParameterizedTest

@CsvSource({

"2,3,5",

"0,0,0",

"-1,1,0"

})

```
void testAdd(int a, int b, int expected) {
```

```
    assertEquals(expected, add(a, b));
```

}

6. Bài tập thực hành

Bài 1

Viết Unit Test cho hàm:

java

String getGreeting(int hour)

- $\text{hour} < 12 \rightarrow \text{"Good Morning"}$
- $12 \leq \text{hour} < 18 \rightarrow \text{"Good Afternoon"}$
- $\text{hour} \geq 18 \rightarrow \text{"Good Evening"}$

Bài 2

Viết Unit Test cho hàm:

java

double getTotal(int quantity, double price)

- $\text{quantity} \geq 10 \rightarrow \text{giảm giá 10\%}$

Bài 3

Viết Unit Test cho hàm:

java

String getTriangleType(int a, int b, int c)

- Tam giác đều
- Tam giác cân
- Tam giác thường
- Không phải tam giác